FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

4

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Student Name	Leandro P. Martinez
Section:	AN43
Professor:	Ms. Crisola Tan
Date:	11/3/2026

I. BACKGROUND INFORMATION

Knowledge Representation (KR) is the method of organizing facts, rules, and relationships so an AI system can understand and reason about the world.

Reasoning (R) allows the system to infer, make decisions, and solve problems based on that knowledge. Two major reasoning approaches from the module:

1. Rule-Based Reasoning (RBR)

Uses IF–THEN rules, a Knowledge Base, Working Memory, and an Inference Engine.

2. Case-Based Reasoning (CBR)

Solves new problems using past cases using the 4R Cycle:

Retrieve → Reuse → Revise → Retain

This laboratory lets you build small working implementations of both reasoning methods.

II. TECHNICAL ASSESSMENT

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Objective

At the end of this laboratory activity, students are expected to:

1. Construct a simple knowledge base using Python data structures.
2. Implement a basic Rule-Based Reasoning (RBR) system using IF–THEN logic.
3. Build a simple Case-Based Reasoning (CBR) system using similarity matching.
4. Compare how RBR and CBR behave when solving the same type of problem.
5. Document observations and insights on KR, RBR, and CBR.

Materials Needed

1. Hardware
   - A laptop or desktop computer
   - Stable internet connection (for research, documentation, submission)
2. Software

A programming environment:

- Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
- Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:

- MS Word, Google Docs, or LibreOffice

- Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

General Instructions

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Include clear comments in your code to describe its logic.
3. Each program should display the correct input, process, and output.
4. Ensure that your script displays clear and readable outputs.
5. Screenshot of program output for RBR
6. Screenshot of program output for CBR
7. Reflection answers
8. Short description of your knowledge base and use case
9. Compile all Python codes and corresponding outputs into one PDF file.
10. Save it as **TA4_lastname1.pdf**
    (Example: TA2_Patilano.pdf)
11. Submit the **python file** and **PDF** file.

LABORATORY TASKS

PART 1 — Knowledge Representation (KR) Setup

Task 1A — Create a Knowledge Base

Create a Python dictionary storing at least:

- 5 facts about an object (e.g., weather, appliances, students, medical symptoms)
- 3 categories (e.g., temperature, status, presence, actions)

Example structure to follow:

```
facts = {
    "temperature": 32,
    "is_raining": False,
    "humidity": 85,
}
```

PART 2 — Implement Rule-Based Reasoning (RBR)

Task 2A — Create at least 3 IF–THEN rules

Write Python functions representing rules.

Example:

```
def rule_hot(facts):
    if facts["temperature"] > 30:
        return "Turn on the aircon"
```

Task 2B — Implement an Inference Engine

Create a loop that:

1. Reads the knowledge base
2. Checks each rule

3. Returns all actions inferred

Task 2C — Display Output

Example [Program should print:]

```
Rule-Based Reasoning Actions:
- Turn on the aircon
- Bring umbrella
```

PART 3 — Implement Case-Based Reasoning (CBR)

Task 3A — Build a Case Base

Create at least **3 past cases**, each containing:

- A set of problem features
- A matching solution

Example:

```
case_base = [
    {"problem": {"won't turn on": True, "noise": False},
     "solution": "Check power cable."}
]
```

Task 3C — Reuse Step

Return the solution of the most similar case.

Task 3D — Revise Step

Allow student to adjust solution manually or return unchanged.

Task 3E — Retain Step

Add the new case to the case base.

PART 4 — Comparative Reflection

Write a short paragraph answering:

1. Which system (RBR or CBR) was easier to implement? Why?
2. Which is more flexible in solving unstructured problems?
3. Which is better for situations with continuously changing data?
4. Examples of real-world applications for both.

- RBR was easier to implement because it used straightforward IF–THEN rules, while CBR required comparing cases, calculating similarity, and revising and storing solutions. CBR is generally more flexible for less structured problems because it can adapt solutions from similar experiences, although it still needs a way to represent and compare problems. CBR is also useful when new cases arrive continuously, provided outdated cases are reviewed; RBR works well when input values change but the rules remain valid. Real-world applications include equipment monitoring and eligibility checks for RBR, and technical support and repair recommendations for CBR.
-

Your Code - Laboratory Tasks ⇩

```
import sys
from copy import deepcopy


facts = {
    "power": {"turns_on": True, "charger_connected": True},
    "cooling": {"overheating": True, "fan_noisy": True},
    "performance": {"runs_slowly": True},
}


def rule_power(problem):
    if (not problem["power"]["turns_on"]
            and not problem["power"]["charger_connected"]):
        return "Connect the charger and check the power connection."


def rule_heat(problem):
    if problem["power"]["turns_on"] and problem["cooling"]["overheating"]:
        return "Shut down, let the laptop cool, and check for blocked
vents."


def rule_fan(problem):
    if problem["power"]["turns_on"] and problem["cooling"]["fan_noisy"]:
        return "Check airflow; have persistent fan noise inspected."


def rule_slow(problem):
    if (problem["power"]["turns_on"]
            and problem["performance"]["runs_slowly"]):
        return "When safe to restart, close unnecessary background apps."
```

```
def infer(problem):
    """Check every rule and collect all matching actions."""
    actions = []
    for rule in [rule_power, rule_heat, rule_fan, rule_slow]:
        action = rule(problem)
        status = "FIRED" if action else "not matched"
        print(f"  {rule.__name__}: {status}")
        if action:
            actions.append(action)
    return actions


case_base = [
    {"id": "C1", "problem": {
        "power": {"turns_on": False, "charger_connected": False},
        "cooling": {"overheating": False, "fan_noisy": False},
        "performance": {"runs_slowly": False}},
     "solution": "Connect the charger and check the power connection."},
    {"id": "C2", "problem": {
        "power": {"turns_on": True, "charger_connected": True},
        "cooling": {"overheating": True, "fan_noisy": True},
        "performance": {"runs_slowly": False}},
     "solution": (
         "Shut down, let the laptop cool, and check for blocked vents. "
         "Have persistent fan noise inspected.")},
    {"id": "C3", "problem": {
        "power": {"turns_on": True, "charger_connected": True},
        "cooling": {"overheating": False, "fan_noisy": False},
        "performance": {"runs_slowly": True}},
       "solution": "Close unnecessary background apps and review startup
apps."},
]


def similarity(new_problem, old_problem):
    """Equal-weight exact matching: matching facts / total facts."""
    matches = 0
```

```
    total = 0
    for category, values in new_problem.items():
        for feature, value in values.items():
            total += 1
            old_values = old_problem.get(category, {})
            if feature in old_values and old_values[feature] == value:
                matches += 1
    return matches / total if total else 0.0


def retrieve(problem, cases):
    """Return the highest-scoring case; ties keep the first case."""
    best_case, best_score = None, -1.0
    for case in cases:
        score = similarity(problem, case["problem"])
        print(f"  {case['id']}: {score:.0%} similarity")
        if score > best_score:
            best_case, best_score = case, score
    return best_case, best_score


def reuse(case):
    return case["solution"]


def revise(solution, replacement=""):
    """A blank replacement keeps the retrieved solution unchanged."""
    return replacement.strip() or solution


def retain(problem, solution, cases):
        """Store  a  separate  copy,  so  later  input  edits  cannot  alter
history."""
    cases.append({"id": f"C{len(cases) + 1}",
                  "problem": deepcopy(problem), "solution": solution})
```

```
def main(demo=False):
    print("TA4 - LAPTOP TROUBLESHOOTING")
    print("INPUT: Current laptop facts")
    for category, values in facts.items():
        print(f"  {category}: {values}")

    print("\nRBR PROCESS: Check every IF-THEN rule")
    actions = infer(facts)
    print("Rule-Based Reasoning Actions:")
        for action in actions or ["No matching rule; seek further
information."]:
        print(f"- {action}")

    cases = deepcopy(case_base)
    print("\nCBR PROCESS: Retrieve -> Reuse -> Revise -> Retain")
    best_case, score = retrieve(facts, cases)
    if best_case is None:
        print("No past cases available.")
        return
    print(f"Retrieve: {best_case['id']} selected ({score:.0%}).")
    solution = reuse(best_case)
    print(f"Reuse: {solution}")
    if demo:
            replacement = (solution + " When safe to restart, close
unnecessary "
                    "background apps.")
        print("Revise: Demo adds an action for the slow performance.")
    else:
        replacement = input("Revised full solution (Enter to keep): ")
    final_solution = revise(solution, replacement)
    print(f"Final CBR solution: {final_solution}")
    before = len(cases)
    retain(facts, final_solution, cases)
         print(f"Retain: Added {cases[-1]['id']}; cases {before} ->
{len(cases)}.")
    print("Retention is in memory for this run; no file is saved.")
```

```python
if __name__ == "__main__":
    main(demo="--demo" in sys.argv)
```

